

Chapter 5: Greedy Algorithms

Lecturer: Jasper Lee

Scribe: Jasper Lee

In this chapter, we introduce the Greedy Algorithm paradigm.

A *greedy algorithm* constructs a (partial) solution iteratively via a sequence of short-sighted decisions or choices, with the hope that it works out at the end.

The above quote/informal definition sounds tentative and hesitant, and indeed that is the intention. It is crucial to understand that *most* greedy algorithms are wrong, however tempting it may be to use one.

Having said that, in the case of optimization problems, greedy algorithms are frequently a decent strategy for *approximation* algorithms, namely algorithms that output solutions that are provably not too much worse than the optimum. Approximation algorithms, however, are generally rather out of scope for this course.

1 Motivating Example: Fractional Knapsack

We start the discussion using the motivating example problem of Fractional Knapsack.

Setting: We are given a knapsack with a maximum weight capacity $C \geq 0$, and a list of n completely divisible items (think liquids). Each item $i \in \{1, \dots, n\}$ has a specific weight $w_i \geq 0$ and a total reward $r_i \geq 0$. We are allowed to take any fractional amount $\rho_i \in [0, 1]$ of item i , which contributes a weight of $\rho_i w_i$ to the knapsack and yields a reward of $\rho_i r_i$. The goal is to maximize the total reward subject to the capacity constraint:

$$\sum_{i=1}^n \rho_i w_i \leq C$$

What are some possible greedy strategies to select items? We might consider greedily filling the knapsack with:

1. The item with the smallest weight w_i .
2. The item with the biggest reward r_i .
3. The item with the highest reward density r_i/w_i .

The first two strategies are incorrect. To see why, consider a knapsack with capacity $C = 100$ and the following three items:

Item	Weight (w_i)	Reward (r_i)	Density (r_i/w_i)
1	100	100	1
2	10	60	6
3	1	0.1	0.1

- **Smallest w_i fails:** Greedily taking the smallest weight first results in taking Item 3, then Item 2, and finally 89/100 of Item 1, yielding a total reward of $0.1 + 60 + 89 = 149.1$.
- **Biggest r_i fails:** Greedily taking the biggest reward first results in taking Item 1, exhausting the knapsack capacity immediately for a total reward of 100.

The optimal approach, instead, is to select based on the highest reward density r_i/w_i . In the above example, the approach would fully take Item 2 and then 90/100 of Item 1, achieving a maximum possible reward of $60 + 90 = 150$.

More formally, the following is an implementation of this correct greedy algorithm.

Algorithm 1 Fractional Knapsack

1. Sort items in decreasing order of reward density r_i/w_i .
 2. total_weight $\leftarrow 0$
 3. for $i \leftarrow 1$ to n
 - (a) if total_weight + $w_i \leq C$:
 - i. Take $\rho_i = 1$
 - ii. total_weight $\leftarrow$ total_weight + w_i
 - (b) else:
 - i. Take $\rho_i = \max(\frac{C - \text{total_weight}}{w_i}, 0)$
 - ii. total_weight $\leftarrow C$
-

1.1 The General Proof Framework

But why is Algorithm 1 actually correct? How do we *prove* that it is correct?

A commonly-held but **wrong** intuition for the correctness of a greedy algorithm is that it makes a series of “locally optimal choices” to yield a “globally optimal” solution.

But there are two important (and in hindsight glaringly obvious) issues with this intuition. First, this intuition only discusses optimization problems, but greedy algorithms (as we will see) can also apply to feasibility/satisfiability problems where the goal is to find *any* solution satisfying a set of constraints. Second, even for optimization problems, think about how we would use induction to prove correctness, where the induction variable is, naturally, the number of choices and commitments made by the greedy algorithm. During the execution of the algorithm, most of the time there will be some partially-constructed solution, which does not necessarily have a “solution value” or “solution quality” associated with it, unlike a fully constructed solution. Because of this, it is quite unclear how one would formalize the way these intermediate solutions relate to “global optimality”, whatever that means.

Instead, the **correct** intuition for greedy algorithms is that:

For every choice the greedy algorithm makes, it did not mess up.

While this intuition sounds informal, we can in fact capture it using induction, with the following proof framework:

- **Induction Variable:** The number of choices made in the algorithm/partial solution.
- **Induction Hypothesis:** For the partial solution constructed from the first i choices of the algorithm, there **exists** an optimal (or feasible) solution that is “compatible” with this partial solution.

The notion of “compatibility” depends on the problem at hand, and it is critical for you as the algorithm designer and analyst to explicitly specify what it means exactly, much like how it is your responsibility to determine and specify what “correct” means.

1.2 Proving Correctness for Fractional Knapsack

Let us now apply the above framework to prove the correctness of Algorithm 1.

Induction Hypothesis: Let $\rho_1, \dots, \rho_i$ be the partial solution constructed at the end of the i^{th} loop iteration. Then, there exists an¹ optimal solution $\rho_1^{(opt)}, \dots, \rho_n^{(opt)}$ such that $\rho_1 = \rho_1^{(opt)}, \dots, \rho_i = \rho_i^{(opt)}$.

For this specific problem and algorithm, the notion of “compatibility” is simple, that an optimal solution includes the choices that the algorithm made.

Base Case ($i = 0$): The partial solution is empty, so it trivially agrees with any optimal solution on the first 0 choices.

Induction Step: Assume there is an optimal solution such that $\rho_1 = \rho_1^{(opt)}, \dots, \rho_i = \rho_i^{(opt)}$. We need to show that there is also an optimal solution compatible additionally with ρ_{i+1} chosen by the algorithm. Note, carefully, that we are allowed to construct a *new* optimal solution as long as it does remain optimal.

Consider the choice ρ_{i+1} made in the $i + 1^{\text{st}}$ loop iteration. In either case of the loop iteration (whether $\rho_{i+1} = 1$ or $\rho_{i+1} < 1$), it must be true that $\rho_{i+1} \geq \rho_{i+1}^{(opt)}$.

If $\rho_{i+1} = \rho_{i+1}^{(opt)}$, then the assumed optimal solution is already compatible with choice $i + 1$, and we are done.

If they are not equal, then we have $\rho_{i+1} > \rho_{i+1}^{(opt)}$. Thus, there will be at least $(\rho_{i+1} - \rho_{i+1}^{(opt)})w_{i+1}$ weight taken by the $\rho^{(opt)}$ solution in items $i + 2$ through n ; otherwise, there is unused capacity in that solution, and we might as well increase $\rho_{i+1}^{(opt)}$ to use up the capacity.

We now construct a new optimal solution $\rho'_1, \dots, \rho'_n$ by taking $(\rho_{i+1} - \rho_{i+1}^{(opt)})w_{i+1}$ weight away from items $i + 2$ through n in the $\rho^{(opt)}$ solution, and moving that weight back to item $i + 1$.

Since the algorithm sorted the items by density, item $i + 1$ has reward density at least that of any item from $i + 2$ to n . Therefore, this new solution ρ' has a total reward that is no worse than $\rho^{(opt)}$, implying that ρ' must also be an optimal solution. By construction, ρ' is equal to the greedy algorithm’s choices up to item $i + 1$: $\rho'_1 = \rho_1, \dots, \rho'_{i+1} = \rho_{i+1}$. This completes the induction step.

¹Note that we use the word “an” not “the”, since there might be multiple optimal solutions, and indeed this is something we consider in the proof.

The above is an instance of a **swapping argument** that is common for analyzing greedy algorithms.

Wrap up: At the end of the n^{th} loop iteration, the induction hypothesis dictates that there exists an optimal solution $\rho^{(opt)}$ such that $\rho_1 = \rho_1^{(opt)}, \dots, \rho_n = \rho_n^{(opt)}$. This implies the algorithm's solution ρ is optimal.

2 Choose Your Team Tournament

We now look at a *feasibility* problem, as opposed to an optimization problem, where being greedy is a correct approach.

Setting: There are two teams, Team A and Team B, each consisting of n members. The members have power levels represented in arrays $A[1..n]$ and $B[1..n]$, which are not necessarily sorted. Each member of Team A will compete in a 1-on-1 match with exactly one member of Team B, where a Team A member wins if and only if their power level is at least the Team B opponent's. Supposing that Team A chooses the matching, is it possible for Team A to win all n matches?

To derive an algorithm, consider trying to find a Team A member to match up with $B[1]$. Which Team A member would be a choice that you *might as well* make? We need to make sure to win that match, so the choice must be a member whose power level is greater than $B[1]$. But which one should we pick? In this case, we might as well pick the weakest such Team A member. And this process can be repeated until all team members are matched, yielding the following algorithm.

Algorithm 2 Choose Your Team Tournament

1. For $i \leftarrow 1$ to n :
 - (a) Find the weakest member $A[j]$ such that $A[j] \geq B[i]$.
 - (b) If such an element exists, match $A[j]$ with $B[i]$ and remove $A[j]$ from consideration.
 - (c) If no such element exists, immediately output “no”.
 2. If the loop completes successfully, output “yes” (and the constructed matching).
-

Runtime Analysis: This can be implemented in $O(n \log n)$ time by judiciously using a binary search tree to store elements of Team A, but we omit the details here.

2.1 Correctness

We want to show that **Algorithm 2** outputs “yes” if and only if there exists a winning matching. If the algorithm terminates and outputs “yes”, then by the algorithm design it explicitly constructed a valid winning matching. It remains to show the converse: If Team A can win, then the algorithm will successfully find a winning matching.

Induction Hypothesis: If Team A can win, then there exists a winning matching that agrees with the matches made by the algorithm in the first i loop iterations.

Base Case ($i = 0$): Trivial, as 0 matches have been made.

Induction Step: Assume there exists a winning matching M that agrees with the algorithm's first i match choices. Now, suppose that in the $i + 1^{\text{st}}$ iteration, the algorithm matches $B[i + 1]$ with $A[j]$.

If M also matches $B[i + 1]$ with $A[j]$, then M already agrees with the first $i + 1$ choices, and we are done.

Otherwise, M matches $B[i + 1]$ with some different element $A[j']$, and M must match $A[j]$ with some element $B[i']$ where $i' > i + 1$ (i' cannot be $< i$ since M matches $B[1..i]$ exactly like the algorithm does).

Note that by the algorithm's greedy criterion, we have $A[j'] \geq A[j]$. Furthermore, since M is a valid winning matching, we know that $A[j'] \geq B[i + 1]$ and $A[j] \geq B[i']$.

We can now construct a new matching M' by swapping these specific assignments: match $A[j]$ with $B[i + 1]$, and match $A[j']$ with $B[i']$. We must verify that this new matching is still winning:

- $A[j] \geq B[i + 1]$: The algorithm explicitly checked this for $A[j]$.
- $A[j'] \geq B[i']$: Since $A[j'] \geq A[j]$ and $A[j] \geq B[i']$, we have $A[j'] \geq B[i']$.

Thus, M' is still a winning matching, and it now agrees with the algorithm on the first $i + 1$ choices. This completes the induction step.

Wrap up: If Team A can win, then by the end of the n^{th} loop iteration, the induction hypothesis guarantees there exists a winning matching that agrees with the algorithm on all n choices, meaning that the loop will have found a winning matching. Thus the algorithm outputs “yes”.

3 Data Compression and Huffman Coding

In this section, we turn our attention to the problem of encoding messages, which are strings of symbols, into binary bit strings. The primary motivations for studying this problem are data compression (specifically, lossless compression) and data transmission (high-voltage vs low-voltage, binary bits). The provably optimal algorithm, the classic Huffman Coding, turns out to be a greedy strategy.

3.1 Problem Setting: Binary and Prefix-Free Codes

Before discussing the algorithm, we first need to introduce the coding setting and a number of technical terms.

- Σ is an *alphabet*, meaning the set of possible message symbols. For example, the set of all English letters is an alphabet. The set $\{0, 1\}$ is the binary alphabet.
- $\{0, 1\}^*$ is the set of all (finite-length) binary bit strings. The $*$ denotes “any possible integer length”.

- A *binary code* is an injective function mapping $\Sigma \rightarrow \{0,1\}^*$, meaning that the code maps every possible symbol into a binary bit string. The binary bit string is also called a *codeword*.

The simplest approach to design a code is to have a *fixed-length code*, where every codeword has the exact same length for every symbol. To encode an alphabet of size $|\Sigma|$, the minimum required length for a fixed-length code is $\lceil \log_2 |\Sigma| \rceil$. On the other hand, a *variable-length code* has no such restriction. For example, consider the alphabet $\Sigma = \{a, b, c, d\}$ and the code $C = \{a \mapsto 0, b \mapsto 10, c \mapsto 110, d \mapsto 111\}$.

Why would we use a variable-length code, say, over the fixed-length code (00, 01, 10, 11)? Suppose we know that the symbol a is much more likely to appear than other symbols, at higher frequency. In such setting, the variable-length code will have much shorter “average” code length or “expected” code length. We will formalize this later.

With variable-length codes, we must be careful to avoid ambiguities during decoding. One way to ensure unique decoding is to use a *prefix-free* code: a (binary) code C such that for every pair of distinct symbols $\sigma, \sigma' \in \Sigma$, the codeword $C(\sigma)$ is not a prefix of the codeword $C(\sigma')$.

Note that fixed-length codes are always trivially prefix-free. The variable-length code C provided in the example above is also prefix-free.

3.2 The Optimal Prefix-Free Binary Code Problem

Let us now formalize the optimization problem for code design. Suppose we are given an alphabet Σ and a probability P_σ associated with each symbol $\sigma \in \Sigma$. Our goal is to compute a prefix-free binary code C that minimizes the expected code length:

$$\min \sum_{\sigma \in \Sigma} P_\sigma |C(\sigma)|$$

where $|C(\sigma)|$ denotes the length of the codeword $C(\sigma)$.

Before designing an algorithm, we make three critical structural observations.

1. Prefix-free binary codes can be represented as binary trees, with symbols $\sigma \in \Sigma$ as leaf nodes, and the edges are labeled with 0 and 1 (ensuring no shared label exists for both edges out of any parent node).

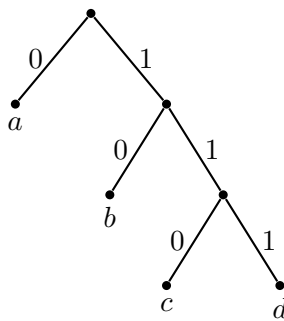


Figure 1: A prefix-free binary code represented as a binary tree. The symbols are located at the leaves.

2. The code length for a symbol σ is exactly the depth of its corresponding leaf node in the tree.
3. Only the tree structure itself matters for the optimization objective; any valid 0/1 labeling of the edges yields the exact same expected code length.

3.3 The Huffman Coding Algorithm

Huffman Coding is a greedy algorithm designed to compute an optimal prefix-free binary code. The algorithm maintains a list of subtrees, where each subtree encodes a disjoint subset of the symbols. It repeatedly merges pairs of subtrees, until a single tree remains: the greedy rule is to always pair two subtrees with the smallest total probabilities.

Algorithm 3 Huffman Coding

1. for $\sigma \in \Sigma$:
 - (a) $T_\sigma \leftarrow$ A single leaf node “ σ ”
 - (b) $P(T_\sigma) \leftarrow P_\sigma$
 2. $List \leftarrow \{T_\sigma\}_{\sigma \in \Sigma}$ // Initialize the set of subtrees
 3. while $List$ size > 1 :
 - (a) $T_1, T_2 \leftarrow$ the subtrees with the smallest and second smallest $P(T)$ in $List$
 - (b) $T_{\text{merge}} \leftarrow \bigwedge_{T_1 \quad T_2}$; $P(T_{\text{merge}}) \leftarrow P(T_1) + P(T_2)$
 - (c) Remove T_1, T_2 from $List$
 - (d) Add T_{merge} back to $List$
 4. Return the final single tree from $List$
-

3.4 Correctness of Huffman Coding

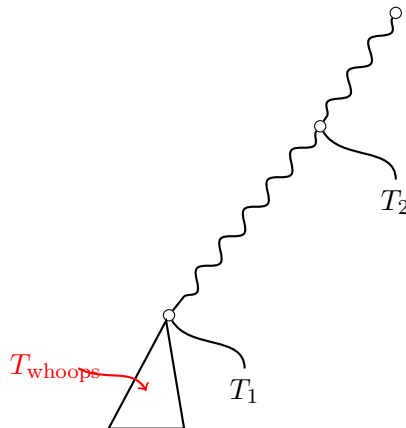
We will prove the correctness of the Huffman coding algorithm using the proof framework we saw earlier.

Induction hypothesis: At the end of the i^{th} iteration, there exists an optimal code tree such that all trees currently in $List$ are subtrees of this optimal tree.

Base case: Before any iterations ($i = 0$), the list consists merely of individual leaves for each symbol in Σ . By definition, any valid code tree must contain these individual leaves, so the base case trivially holds.

Inductive step: Let OPT_i be the optimal code tree guaranteed by the hypothesis, which contains all subtrees in $List$ at the beginning of the $i + 1^{\text{st}}$ iteration. During the $i + 1^{\text{st}}$ iteration, suppose the algorithm merges T_1 and T_2 into T_{merge} . If T_{merge} is already a subtree of OPT_i (meaning T_1 and T_2 are siblings in OPT_i), then we are done.

Otherwise, we assume without loss of generality that T_1 is positioned deeper (no shallower) in OPT_i than T_2 . Since T_1 is not paired with T_2 , its sibling in OPT_i must be some other subtree, which we will call T_{whoops} .



Notice that T_{whoops} must contain at least one subtree in $List$, and it cannot be T_1 or T_2 . By the greedy criterion, T_1 and T_2 have the smallest probabilities in the list, so $P(T_{\text{whoops}}) \geq P(T_2)$.

We modify OPT_i by swapping the positions of T_{whoops} and T_2 . As T_1 is no shallower than T_2 , swapping T_2 into T_{whoops} 's deeper position and pulling T_{whoops} up does not increase the overall expected code length. In other words, the new tree has at most the expected code length as OPT_i , meaning the new tree is also optimal. Furthermore, this new optimal tree has $\bigwedge_{T_1 T_2}$ as a subtree, completing the induction step.

3.5 Huffman Code Length and Discrete Entropy

We have proven that Huffman coding produces the shortest possible prefix-free binary code, but what is its actual expected code length? To answer this, we introduce the notion of discrete entropy.

Definition 5.1 (Discrete Entropy). Consider a discrete random variable X over a domain $\mathcal{X}$. The discrete entropy of X is defined as:

$$H(X) = - \sum_{x \in \mathcal{X}} p_x \log p_x$$

The logarithm is typically base-2 (for binary encodings), which we use here, but other logarithm bases can be meaningful in other settings.

This definition is relevant due to Claude Shannon's Source Coding Theorem.

Theorem 5.2 (Shannon's Source Coding Theorem, Informal). *No lossless coding scheme can encode a random variable X using fewer than $H(X)$ bits on average over n independent and identically distributed symbols, as $n \rightarrow \infty$.*

The proof of this theorem is graduate-level and requires semi-advanced intuition for probability. However, we can state a concrete upper bound for the Huffman Code's expected length relative to the Shannon entropy limit.

Theorem 5.3. *Let $C(X)$ be a Huffman code for random variable X . Then the expected code length satisfies:*

$$\mathbb{E}_X[|C(X)|] \leq H(X) + 1$$

The proof is also challenging, and I am unaware of any direct analysis of Huffman Code showing this bound. Instead, typical proofs show that a different prefix-free coding scheme, called the Shannon-Fano code, achieves this $H(X) + 1$ upper bound. Huffman Code's performance then follows from its optimality among all prefix-free codes.

4 Minimum Spanning Trees

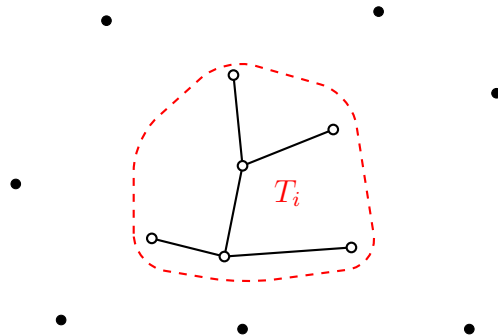
Setting: Given a connected, weighted, undirected graph, the goal is to compute a spanning tree with the minimum total edge weight. Recall that a spanning tree is a tree connecting all vertices, hence with $n - 1$ edges.

There are two standard greedy algorithms for this problem: Prim's algorithm and Kruskal's algorithm. For this course, we only have time to cover Prim's algorithm.

4.1 Prim's Algorithm

The core idea is to grow a spanning tree outward from a single vertex, adding one edge at a time. Let us try deriving the algorithm.

Consider some intermediate stage where we have a partial tree T_i . What does any spanning tree need to do? It needs to connect T_i with *some* vertex outside of T_i .



The key observation is that we *may as well* take the minimum-weight edge between T_i and $V \setminus T_i$.

Algorithm 4 Prim's Algorithm

1. $Connected \leftarrow \{s\}$ // s arbitrarily chosen
 2. $T \leftarrow \emptyset$ // empty tree, no edges
 3. while there exists edge between $Connected$ and $V \setminus Connected$:
 - (a) Let $(u, v) = \underset{\substack{u \in Connected \\ v \notin Connected \\ (u, v) \in E}}{\arg \min} w_{(u, v)}$
 - (b) Add (u, v) to T
 - (c) Add v to $Connected$
 4. return T
-

4.2 Correctness

We can again prove the correctness of Prim's algorithm using the framework introduced earlier.

Induction hypothesis: At the end of the i^{th} iteration, there exists a minimum spanning tree T_i^{opt} such that T_i^{opt} contains all edges in T .

Base case: $i = 0$, trivial.

Induction step: Assume the induction hypothesis for iteration i , so there is a minimum spanning tree T_i^{opt} containing all of T_i , where T_i is the state of T at end of iteration i .

Call (u, v) the new edge added in iteration $i+1$, with $u \in T_i$ and $v \notin T_i$. If $(u, v) \in T_i^{opt}$, then we are done (explicitly, let $T_{i+1}^{opt} = T_i^{opt}$). Otherwise, $(u, v) \notin T_i^{opt}$, so $T_i^{opt} \cup \{(u, v)\}$ has a cycle.

At this point, we need to identify some component (really, edge) in T_i^{opt} to swap with (u, v) . Now the claim is that there must exist some edge $(a, b) \neq (u, v)$ on that cycle where $a \in T_i$ and $b \notin T_i$. Why? Consider walking along the cycle from u , starting from $u \rightarrow v$. This edge goes from T_i to $V \setminus T_i$. But eventually, along the walk, we end up back at u which is in T_i . Therefore, along the way there must be some $(a, b) \neq (u, v)$ in the cycle such that $a \in T_i$ and $b \notin T_i$.

To construct T_{i+1}^{opt} , we swap (a, b) with (u, v) , namely

$$T_{i+1}^{opt} = T_i^{opt} \cup \{(u, v)\} \setminus \{(a, b)\}$$

By the greedy criterion of the algorithm, we know that $w_{(a, b)} \geq w_{(u, v)}$, implying that $w(T_{i+1}^{opt}) \leq w(T_i^{opt})$. Furthermore, T_{i+1}^{opt} still has $n - 1$ edges and connects all vertices, hence is a spanning tree. Therefore, T_{i+1}^{opt} is another minimum spanning tree, which by construction contains T_{i+1} as desired.

Wrap up: Upon termination, T connects all vertices reachable from s , and there exists a minimum spanning tree containing all of T . So T is a minimum spanning tree.

Algorithm 5 Prim's Algorithm via Heap

1. $Connected \leftarrow \{s\}; T \leftarrow \emptyset; H \leftarrow$ empty heap
 2. $Pointers \leftarrow$ array indexed by V , initialized to NULL
 3. for every $v \neq s$:
 - (a) if $(s, v) \in E$ then $key(v) \leftarrow w_{(s,v)}$ else $key(v) \leftarrow \infty$
 - (b) $edge(v) \leftarrow (s, v)$
 - (c) Create $(key(v), (v, edge(v)))$ with pointer p_v , add to H and store p_v in $Pointers[v]$
 4. while H is non-empty:
 - (a) $(key(u), (u, edge(u))) \leftarrow \text{EXTRACTMIN}(H)$
 - (b) Add u to $Connected$, add $edge(u)$ to T
 - (c) for each edge (u, v) where $v \notin Connected$:
 - i. if $key(v) > w_{(u,v)}$:
 - A. DELETE v from H using $Pointers[v]$
 - B. $key(v) \leftarrow w_{(u,v)}; edge(v) \leftarrow (u, v)$
 - C. Create entry $(key(v), (v, edge(v)))$ for v in H with pointer p_v , update $Pointers[v]$
 5. return T
-

4.3 Implementation

The algorithmically challenging part is finding the minimum outgoing edge from T efficiently. We can use a heap data structure to achieve this goal.

Heaps support the following operations:

- $\text{INSERT}(x = (\text{KEY}, \text{VALUE}))$: adds x into the data structure.
- $\text{EXTRACTMIN}()$: returns an element with the minimum key and deletes it from the data structure.
- $\text{DELETE}(\text{POINTER TO ELEMENT } x)$: deletes a specific element x from the data structure.

Fact: There is a heap implementation such that all operations take $O(\log n)$ time, where n is the number of elements in the heap at the time.

For Prim's algorithm, we can use such a heap to store vertices to decide which one to grow the tree to. See [Algorithm 5](#), which runs in $O((|V| + |E|) \log |V|)$ time.

To reason about data structures (and this is the only small part of the course where we can afford the time to discuss how to reason about them), it is crucial to specify an invariant that keeps holding true no matter what sequence of allowable operations is made to the structure.

Invariant: At the end of every loop iteration, the heap H contains all vertices $v \in V \setminus \text{Connected}$ with key $\min_{u \in \text{Connected}} w(u, v)$.

5 Dijkstra's Algorithm

We now return to the single-source shortest path problem. Previously, in the dynamic programming chapter, we covered the Bellman-Ford algorithm which runs in $O(|V||E|)$ time, while allowing negative weights and even detects the existence of negative cycles reachable from the source vertex. Now we introduce Dijkstra's algorithm, another classic greedy algorithm, which by contrast can be implemented to run in $O((|V| + |E|) \log |V|)$ time, but it assumes non-negative edge weights. The structure of Dijkstra's algorithm, as we will see, is remarkably similar to Prim's algorithm.

Note Our presentation of Dijkstra's algorithm will be somewhat different from standard sources, textbook or otherwise. Standard sources tend to describe and analyze Prim's and Dijkstra's algorithms significantly differently from each other, even though they are fundamentally very similar. The reformulation in this section is designed to unify their treatment under the same analysis framework introduced at the beginning of the chapter.

5.1 Shortest Path Tree

Prim's algorithm computes a minimum spanning tree, and we need an analogous notion for the shortest path problem in order to derive Dijkstra's algorithm.

Definition 5.4 (Shortest path tree). A directed tree T is a *shortest path tree* with respect to a directed graph G and source vertex s if:

1. T contains all reachable vertices from s ,
2. T is a subgraph of G , and
3. For all such vertices v , the path $s \rightsquigarrow_T v$ in T is a shortest path in G .

A priori (and without relying on a shortest path algorithm), it might not even be obvious that shortest path trees should exist, so we first show that here, before deriving Dijkstra's algorithm to compute one.

Existence: For any vertex v reachable from s , choose any shortest path $s \rightsquigarrow_G v$ with the fewest hops. Let $\text{prev}(v)$ be the vertex before v on this path, and let $\text{hop}(v)$ be the number of hops on the chosen path.

Observe that $\text{hop}(\text{prev}(v)) = \text{hop}(v) - 1$ (this is left as an exercise to prove for the reader, leveraging the fact that there are no negative-weight edges).

Finally, we can construct a shortest path tree hop-by-hop, adding edges $\text{prev}(v) \rightarrow v$ for every $v \neq s$. By construction, there are $n - 1$ edges added with no cycles, so the graph is indeed a tree. We leave this as an exercise to show that indeed the tree is a shortest path tree, by induction on the number of hops.

5.2 Algorithm

The idea of Dijkstra's algorithm is very similar to Prim's, to grow a shortest path tree from source s . Consider again, like in our discussion for Prim's algorithm, some intermediate stage where we have a partial tree T_i (which, eventually by induction, will be guaranteed to be a subtree of some shortest path tree). A shortest path tree, being a spanning tree, again needs to connect a vertex in T_i to some vertex outside of T_i . Which edge should we pick at the boundary of T_i to grow it into T_{i+1} ? Dijkstra's algorithm greedily picks the edge $u \rightarrow v$ so that the shortest path from $s \rightsquigarrow_{T_i} u$ plus $w_{u \rightarrow v}$ is the shortest, as follows.

Algorithm 6 Dijkstra's Algorithm

1. $Connected \leftarrow \{s\}$
 2. $T \leftarrow \emptyset$
 3. while there exists an edge from $Connected$ to $V \setminus Connected$:
 - (a) Find $(u \rightarrow v) = \arg \min_{\substack{u \in Connected \\ v \notin Connected \\ (u \rightarrow v) \in E}} dist_T(s, u) + w_{u \rightarrow v}$
 - (b) Add $u \rightarrow v$ to T and v to $Connected$.
 4. return T
-

Exercise: Figure out how to implement this in $O((|V| + |E|) \log |V|)$ time using the heap data structure.

5.3 Correctness

Given the similarity to Prim's algorithm, we can prove the correctness of Dijkstra's analogously as well.

Induction hypothesis: At the end of every iteration i , T_i (the state of T after iteration i) is such that

1. $Connected_i$ (the state of $Connected$ after iteration i) is the set of v reachable from s in T_i , and the number of edges in T_i is exactly $|Connected_i| - 1$.
2. There exists a shortest path tree T_i^{opt} containing all edges in T_i .

A corollary of the second point is that $dist_{T_i}(s, v) = dist_{T_i^{opt}}(s, v)$. The $\geq$ direction is because T_i^{opt} is a shortest path tree, and the $\leq$ direction is because T_i^{opt} contains all of T_i and so the paths of T_i are shortest paths.

Induction step:

1. Condition 1 is trivial. Every v we add is reachable from s .
2. Suppose T_i^{opt} is a shortest path tree containing all of T_i by induction.
Let $u \in T_i \rightarrow v \notin T_i$ be the edge chosen in iteration $i + 1$.

If $u \rightarrow v$ is already in T_i^{opt} , then let $T_{i+1}^{opt} = T_i^{opt}$ and Condition 2 holds.

Otherwise, suppose $u \rightarrow v$ is not in T_i^{opt} . Then in T_i^{opt} , the path $s \rightsquigarrow v$ must contain some edge $a \rightarrow b$ where $a \in T_i$ and $b \notin T_i$, and by assumption $a \rightarrow b$ cannot be $u \rightarrow v$. So the path looks like $s \rightsquigarrow a \rightarrow b \rightsquigarrow prev(v) \rightarrow v$.

Observe that

$$\begin{aligned}
 dist_{T_i^{opt}}(s, v) &\geq dist_{T_i^{opt}}(s, a) + w_{a \rightarrow b} && \text{(No negative weight edges)} \\
 &= dist_{T_i}(s, a) + w_{a \rightarrow b} \\
 &\geq dist_{T_i}(s, u) + w_{u \rightarrow v} && \text{(Greedy criterion)}
 \end{aligned}$$

With this, we construct T_{i+1}^{opt} by removing $prev(v) \rightarrow v$ (which cannot be in T_i) and adding $u \rightarrow v$. Reachability is preserved, and all the path distances are no longer than in T_i^{opt} . Thus T_{i+1}^{opt} satisfies Condition 2.

Wrap up: At termination, there is some shortest path tree T^{opt} containing T , and they have the same number of vertices, meaning that the edge count is the same ($|Connected| - 1$). Thus, they are the same tree.